Минобрнауки Российской Федерации
Федеральное государственное образовательное учреждение высшего профессионального образования
Белгородский государственный технологический университет им В. Г. Шухова

Кафедра программного обеспечения вычислительной техники и автоматизированных систем.



Лабораторная работа 1

Дисциплина: Тестирование программных систем

Тема: Составление тест-кейсов для готового кода. Качество ПО и место для тестирования в жизненном цикле.

Выполнил:
Ст. группы ПВ-233
Коровин Александр

Принял: Доденко Олег Павлович

Белгород 2026

Цель работы: сформировать системное представление о качестве ПО и роли тестирования в жизненном цикле; приобрести практические навыки проектирования тест-кейсов и написания автотестов на основе готового кода.

Задачи:

- Проанализировать готовый модуль и выделить бизнес-правила, входные и выходные данные, ограничения.
- Составить набор тест-кейсов: позитивных, негативных, граничных, покрывающих эквивалентные классы.
- Оформить тест-кейсы по стандартным атрибутам.
- Реализовать автотесты на Python + pytest.
- Запустить тесты и зафиксировать результаты.
- Сделать вывод о связи тестирования, качества ПО и жизненного цикла.

Задание:

1. Изучить исходный код и выписать все бизнес-правила.
2. Определить:
   - входные параметры;
   - выходной параметр;
   - допустимые и недопустимые значения;
   - граничные значения.
3. Составить не менее 10 тест-кейсов:
   - позитивные;
   - негативные;
   - граничные;
   - проверяющие скидку для premium-пользователей;
   - проверяющие бесплатную доставку.
4. Написать автотесты на pytest.
5. Запустить тесты командой pytest -v.
6. Сделать вывод: какие ветви кода покрыты, какие риски остались, почему тестирование важно для качества.

1. Анализ исходного кода

Объектом тестирования является функция `calculate_delivery_cost()`, предназначенная для расчёта стоимости доставки в зависимости от суммы заказа, расстояния и наличия премиум-статуса у клиента.

Функция принимает три входных параметра:

- `order_amount` — сумма заказа;
- `distance_km` — расстояние доставки в километрах;
- `is_premium` — логический признак премиум-статуса клиента.

Результатом выполнения функции является стоимость доставки типа `float`. Если сумма заказа или расстояние имеют отрицательное значение, функция возбуждает исключение `ValueError`.

В процессе анализа исходного кода были выделены следующие бизнес-правила:

1. При отрицательной сумме заказа возникает исключение `ValueError`.
2. При отрицательном расстоянии доставки возникает исключение `ValueError`.
3. При сумме заказа от 5000 включительно доставка бесплатная.
4. При расстоянии до 5 км включительно стоимость доставки составляет 200.
5. При расстоянии свыше 5 км и до 20 км включительно стоимость составляет 500.
6. При расстоянии свыше 20 км стоимость рассчитывается по формуле:

$$C = 500 + (d - 20) * 30$$

где d — расстояние доставки в километрах.
7. Для премиум-клиентов применяется скидка 50% на рассчитанную стоимость доставки.

1.1. Исходный код программы

```python
def calculate_delivery_cost(
    order_amount: float,
    distance_km: float,
    is_premium: bool
) -> float:
    if order_amount < 0:
        raise ValueError("order_amount must be >= 0")

    if distance_km < 0:
        raise ValueError("distance_km must be >= 0")

    if order_amount >= 5000:
        cost = 0.0
    elif distance_km <= 5:
        cost = 200.0
    elif distance_km <= 20:
        cost = 500.0
```

```
    else:
        cost = 500.0 + (distance_km - 20) * 30.0

    if is_premium:
        cost *= 0.5

    return cost
```

Листинг 1 — Исходный код функции расчёта стоимости доставки.

2. Определение входных данных и граничных значений

Для проектирования тест-кейсов были рассмотрены допустимые и недопустимые значения входных параметров. Особое внимание уделено границам тарифных диапазонов, поскольку ошибки часто возникают при обработке значений на границе условий.

Таблица 1 — Входные параметры и граничные значения

Параметр	Назначение	Допустимые значения	Недопустимые значения	Граничные значения
`order_amount`	Сумма заказа	Числа от 0 и выше	Отрицательные числа	0; 4999,99; 5000
`distance_km`	Расстояние доставки	Числа от 0 и выше	Отрицательные числа	0; 5; 5,01; 20; 20,01
`is_premium`	Премиум-статус	`True`, `False`	Другие типы отдельно не проверялись	`True`, `False`

В качестве основных техник тест-дизайна использовались классы эквивалентности и анализ граничных значений. Также были предусмотрены позитивные тесты, проверяющие корректные расчёты, и негативные тесты, проверяющие обработку недопустимых входных данных.

3. Разработка тест-кейсов

Для проверки корректности работы программы был разработан набор из 18 автоматических тестовых случаев. Они охватывают расчёт бесплатной доставки, различные тарифные диапазоны, премиум-скидку, граничные значения и обработку отрицательных параметров.

Таблица 2 — Тест-кейсы функции `calculate_delivery_cost()`

ID	Входные данные: сумма, расстояние, premium	Ожидаемый результат	Фактический результат
TC-01	5000; 10; False	0,0	Пройден
TC-02	6000; 30; False	0,0	Пройден
TC-03	4999,99; 10; False	500,0	Пройден
TC-04	1000; 0; False	200,0	Пройден
TC-05	1000; 5; False	200,0	Пройден

TC-06	1000; 5,01; False	500,0	Пройден
TC-07	1000; 20; False	500,0	Пройден
TC-08	1000; 20,01; False	500,3	Пройден
TC-09	1000; 30; False	800,0	Пройден
TC-10	1000; 10; True	250,0	Пройден
TC-11	1000; 5; True	100,0	Пройден
TC-12	1000; 30; True	400,0	Пройден
TC-13	5000; 10; True	0,0	Пройден
TC-14	0; 10; False	500,0	Пройден
TC-15	−1; 10; False	`ValueError`	Пройден
TC-16	−0,01; 10; False	`ValueError`	Пройден
TC-17	1000; −1; False	`ValueError`	Пройден
TC-18	1000; −0,01; False	`ValueError`	Пройден

Результаты в последнем столбце соответствуют фактическому запуску автоматических тестов. Все 18 тестовых случаев завершились успешно.

4. Реализация автоматических тестов

Для автоматизации проверки использовалась библиотека `pytest`. Тесты размещены в отдельном файле `test_delivery.py`.

Параметризация `pytest.mark.parametrize` позволяет проверять функцию на нескольких наборах входных данных. Для проверки исключений применяется `pytest.raises(ValueError)`. При сравнении чисел с плавающей точкой используется `pytest.approx()`, позволяющий учитывать возможные погрешности вычислений.

4.1. Код автоматических тестов

```
import pytest
from delivery import calculate_delivery_cost


@pytest.mark.parametrize(
    "amount, distance, premium, expected",
    [
        # Бесплатная доставка при заказе от 5000
        (5000, 10, False, 0.0),
        (6000, 30, False, 0.0),

        # Проверка границ суммы заказа
        (4999.99, 10, False, 500.0),

        # Расстояние до 5 км включительно
        (1000, 0, False, 200.0),
        (1000, 5, False, 200.0),
        (1000, 5.01, False, 500.0),
```

```
        # Расстояние до 20 км включительно
        (1000, 20, False, 500.0),
        (1000, 20.01, False, 500.3),

        # Расстояние больше 20 км
        (1000, 30, False, 800.0),

        # Премиум-клиент получает скидку 50%
        (1000, 10, True, 250.0),
        (1000, 5, True, 100.0),
        (1000, 30, True, 400.0),

        # Бесплатная доставка и премиум
        (5000, 10, True, 0.0),

        # Нулевая сумма заказа
        (0, 10, False, 500.0),
    ],
)
def test_delivery_cost(amount, distance, premium, expected):
    result = calculate_delivery_cost(amount, distance, premium)
    assert result == pytest.approx(expected)


@pytest.mark.parametrize("amount", [-1, -0.01])
def test_negative_order_amount(amount):
    with pytest.raises(ValueError, match="order_amount"):
        calculate_delivery_cost(amount, 10, False)


@pytest.mark.parametrize("distance", [-1, -0.01])
def test_negative_distance(distance):
    with pytest.raises(ValueError, match="distance_km"):
        calculate_delivery_cost(1000, distance, False)
```

Листинг 2 — Автоматические тесты функции расчёта стоимости доставки.

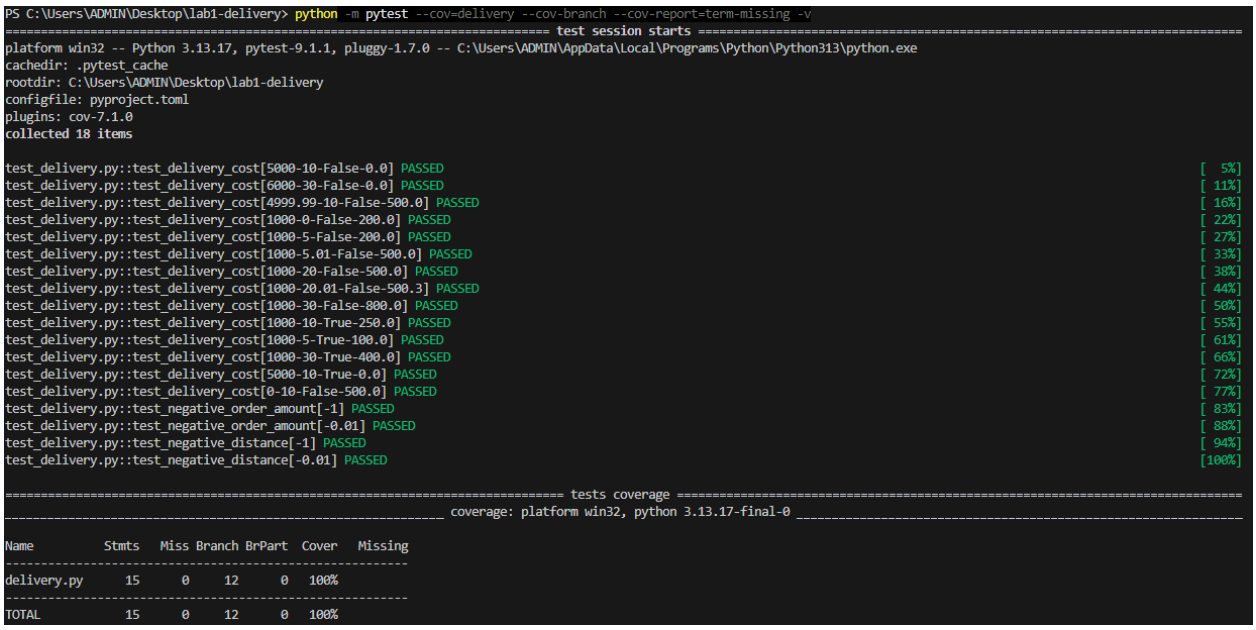
4.2. Запуск тестов

Для выполнения тестов использовалась команда:

```
python -m pytest -v
```

Для измерения покрытия кода применялась команда:

```
python -m pytest --cov=delivery --cov-branch --cov-report=term-missing -v
```



Скриншот 1 — Результаты выполнения автоматических тестов.

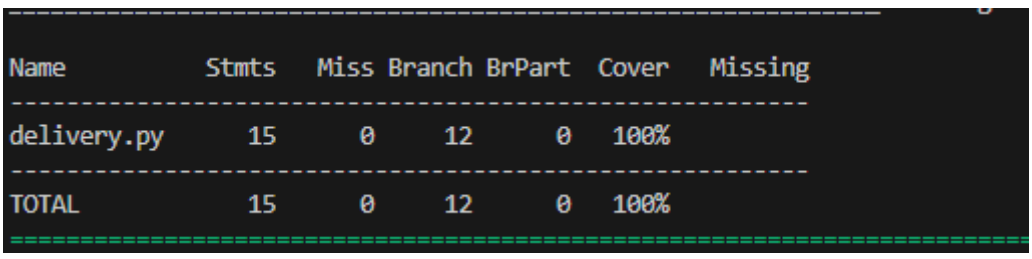
5. Результаты тестирования и покрытие кода

По результатам запуска автоматических тестов все 18 проверок завершились успешно. Ошибок выполнения не обнаружено.

Инструмент `pytest-cov` показал 100% покрытия учитываемых операторов и ветвей кода модуля `delivery.py`.

Таблица 3 — Результаты измерения покрытия кода

Показатель	Результат
Количество автоматических тестов	18
Успешно пройденные тесты	18
Неуспешные тесты	0
Учитываемые операторы (`Stmts`)	15
Непокрытые операторы (`Miss`)	0
Учитываемые ветви (`Branch`)	12
Частично покрытые ветви (`BrPart`)	0
Общее покрытие (`Cover`)	100%



Скриншот 2 — Результаты измерения покрытия кода с помощью pytest-cov.

5.1. Анализ покрытия ветвей

Таблица 4 — Проверенные условия и ветви кода

№	Проверяемое условие	Тестовые случаи	Результат
1	Отрицательная сумма заказа	TC-15, TC-16	Покрыто
2	Отрицательное расстояние	TC-17, TC-18	Покрыто
3	Бесплатная доставка при сумме от 5000	TC-01, TC-02, TC-13	Покрыто
4	Расстояние до 5 км включительно	TC-04, TC-05, TC-11	Покрыто
5	Расстояние свыше 5 км и до 20 км	TC-03, TC-06, TC-07, TC-10	Покрыто
6	Расстояние свыше 20 км	TC-08, TC-09, TC-12	Покрыто
7	Применение премиум-скидки	TC-10, TC-11, TC-12	Покрыто
8	Отсутствие премиум-скидки	TC-01–TC-09, TC-14	Покрыто

Полученные показатели подтверждают, что все учитываемые инструментом строки и ветви модуля были выполнены в ходе тестирования. При этом 100% покрытия не означает, что программа гарантированно не содержит ошибок: тесты проверяют только предусмотренные сценарии и выбранные значения входных данных.

6. Роль тестирования в жизненном цикле ПО

Тестирование является важной частью жизненного цикла программного обеспечения. Оно помогает проверять соответствие реализации требованиям, обнаруживать ошибки и снижать риск появления дефектов при эксплуатации программы.

Разработка тест-кейсов позволяет систематически проверять бизнес-правила, а автоматизированные тесты обеспечивают возможность повторного выполнения проверок после изменений кода. Раннее обнаружение ошибок упрощает их исправление и может уменьшить затраты на сопровождение программного обеспечения.

Измерение покрытия помогает оценить, какие строки и ветви кода выполнялись во время тестирования. Однако показатель покрытия необходимо рассматривать вместе с качеством самих тест-кейсов и полнотой проверяемых требований.

7. Вывод

В ходе лабораторной работы был проанализирован исходный код функции `calculate_delivery_cost()`, определены её входные параметры, бизнес-правила и граничные значения. На основе анализа были разработаны и реализованы 18 автоматических тестов с использованием библиотеки `pytest`.

Все 18 тестов завершились успешно. С помощью `pytest-cov` установлено, что покрытие учитываемых операторов и ветвей модуля `delivery.py` составляет 100%, непокрытых операторов и частично покрытых ветвей не обнаружено.

В результате работы были получены практические навыки проектирования тест-кейсов, автоматизации проверок и анализа покрытия кода. Проведённое тестирование подтверждает корректность функции для проверенных сценариев, но не исключает

необходимости дальнейших проверок некорректных типов данных, дополнительных граничных значений и других потенциальных рисков.